CAPTURA DE TRÁFEGO: TCPDUMP E TSHARK

O log da aplicação dizia "falha na requisição"; o da API não registrava requisição alguma. ***As duas equipes estavam certas*** *— o problema estava no fio. Um* `tcpdump` *mostrou três* `[S]` *saindo e nenhum* `[S.]` *voltando:* ***quando os logs de ponta discordam, o tráfego é o árbitro que não mente.***

084
GUIAS LINUX

O QUE A CAPTURA MOSTRA — E A FRONTEIRA EXATA ONDE O TLS COMEÇA

`cab. Ethernet` — MAC de origem e destino	VISÍVEL
`cab. IP` — **quem fala com quem**, endereços de origem e destino	VISÍVEL
`cab. TCP` — portas, flags, número de sequência	VISÍVEL
`TLS ClientHello` — **SNI: o nome do site visitado**, ainda em claro	VISÍVEL metadado
`dados HTTP` — URL, cabeçalhos, cookies, corpo, senha	CIFRADO

O TLS protege o conteúdo, não a existência da conversa. Sem quebrar cifra nenhuma, uma captura revela **quem** falou com **quem**, **quando**, **quanto** e — pelo SNI — **qual site**. Para mapear infraestrutura, detectar exfiltração ou identificar comunicação com um servidor de controle, o metadado basta. Conteúdo em claro só aparece em protocolo sem TLS (HTTP simples, FTP, Telnet, DNS tradicional) — daí o valor num diagnóstico e o risco numa rede hostil.

LER AS FLAGS: O HANDSHAKE E OS DOIS PADRÕES DE FALHA

`[S]` SYN · abrindo | `[S.]` SYN-ACK · aceitando | `[.]` ACK · confirmação | `[P.]` PSH-ACK · dado entregue | `[F.]` FIN-ACK · fecha ordenado | `[R]` RST · recusa ou aborto

SAUDÁVEL	`[S]` → `[S.]` → `[.]` — o handshake de três vias completo. Depois vem `[P.]` (dados) e `[F.]` no fim. Handshake completo seguido de silêncio já **não é problema de rede**: é a aplicação.
SÓ SYN, REPETIDO	`[S]` `[S]` `[S]` sem nenhum `[S.]` — **ninguém respondeu**. Firewall descartando em silêncio ou destino fora do ar. Foi o caso da abertura.
RESET IMEDIATO	`[S]` → `[R.]` — **recusa ativa**: a máquina está viva e respondeu dizendo que ninguém escuta naquela porta. Silêncio é firewall; `[R]` é porta fechada em host vivo.

TCPDUMP CAPTURA, TSHARK ANALISA

kali@kali: ~

```
$ sudo tcpdump -i any -n 'host 10.0.0.80 and port 443'
10.0.0.20.52014 > 10.0.0.80.443: Flags [S], seq 998...
10.0.0.20.52014 > 10.0.0.80.443: Flags [S], seq 998...
10.0.0.20.52014 > 10.0.0.80.443: Flags [S], seq 998...
# -n NÃO resolve nomes: a resolução reversa gera tráfego DNS
# que polui a própria captura e a deixa lentíssima.
$ sudo tcpdump -i any -n -w caso.pcap host 10.0.0.80
# grava o tráfego bruto; analise depois, com calma
$ tshark -r caso.pcap -Y 'http.request.method == "POST"' \
    -T fields -e ip.src -e http.host -e http.request.uri
$ tshark -r caso.pcap -q -z conv,tcp   # conversas
$ tshark -r caso.pcap -q -z io,phs     # protocolos
# capturar no servidor e ver no desktop, num cano só:
$ ssh srv 'sudo tcpdump -i any -U -w - -n not port 22' \
    | wireshark -k -i -
```

BPF: O FILTRO QUE SALVA A CAPTURA

eixos	`host`/`net` · `src`/`dst` · `tcp`/`udp` `port`, combinados com **`and`**, **`or`**, **`not`**
exemplo	`'tcp and not port 22'` — tudo menos o seu próprio SSH
só SYN	`'tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn'`: quem tenta conectar aonde
só RST	`'tcp[tcpflags] & tcp-rst != 0'`: conexões recusadas

Filtro de captura (BPF) e filtro de exibição são linguagens diferentes. O BPF (`tcp port 443`, no `tcpdump` e no `tshark -f`) decide o que **entra** na captura, e roda no kernel — barato mesmo sob carga. O de exibição (`http.request.method == "POST"`, no `-Y`) filtra o que **já foi** capturado e é muito mais expressivo. Trocar um pelo outro é o erro mais comum, e a mensagem de erro não deixa óbvio qual foi.

Capture apenas em rede própria ou com autorização escrita. Em rede de terceiros, o mesmo comando é interceptação de comunicação alheia — com o enquadramento da Lei 12.737/2012 e da legislação de sigilo das comunicações. A diferença entre auditoria e crime não está no binário, está na autorização. **E um `.pcap` é dado sensível de altíssimo valor:** tráfego sem TLS carrega senha, cookie de sessão e conteúdo privado em claro. Trate-o como um dump de credenciais — `umask 077`, cifra em repouso, `shred -u` ao descartar. Um `.pcap` esquecido em `/tmp` é vazamento.

RESUMO RÁPIDO

- ✓ **Quando os logs das duas pontas discordam, a captura é o árbitro** — ela mostra o que de fato passou no fio.
- ✓ **O TLS esconde o conteúdo, não a conversa:** endereços, portas, tamanhos, tempos e o SNI ficam visíveis.
- ✓ `[S]` **sem** `[S.]` **é firewall ou host morto;** `[R]` **é recusa ativa** num host vivo. Reconhecer os dois resolve a maioria dos casos.
- ✓ `-n` **sempre:** resolver nomes polui a captura com o próprio DNS e a deixa lentíssima.
- ✓ `tcpdump -w` **captura, Wireshark/** `tshark` **analisa** — e `-T fields -e` transforma a captura numa fonte tabular para `awk` e `sort`.
- ✓ **Filtro estreito desde o início:** arquivo pequeno, análise focada e nada capturado além do necessário.

NO KALI

A captura é reconhecimento passivo puro: em rede autorizada, o `tcpdump` mapeia hosts, serviços e padrões de comunicação **sem enviar um único pacote** — invisível para o alvo. A legalidade é definida pela rede, não pela ferramenta. **Modo promíscuo não vence a comutação:** numa rede com switch a interface só vê o que é dela e o broadcast, então capturar o tráfego de outra máquina exige espelhamento de porta, um *tap* ou um ataque ativo de camada 2 (ARP spoofing) — que é intrusivo e já não é passivo. Em Wi-Fi aberto o modo monitor vê tudo do ar; em Wi-Fi cifrado, só metadados de rádio. **A captura é a prova contra o host que mente:** um `ss` ou um `iptables -L` rodados numa máquina comprometida podem ter sido adulterados por um rootkit de espaço de usuário — a captura feita **de fora** vê o tráfego real, incluindo a conexão de saída para o servidor de controle que o host esconde. É por isso que investigação séria de comprometimento não confia só no que a máquina relata sobre si. **Filtro estreito é discrição, não só conveniência:** capturar tudo num engajamento gera arquivo gigante cheio de dados de terceiros possivelmente fora do escopo — um `host <alvo> and port <serviço>` coleta só o que o contrato permite. **RHEL:** o `tshark` vem em `wireshark-cli`, a captura sem root é gerida pelo grupo `wireshark`, e o SELinux pode negar a escrita do `.pcap` fora de `/tmp` ou do home sem dizer que é SELinux.